

Follow-up Meeting

Anonymous Credentials

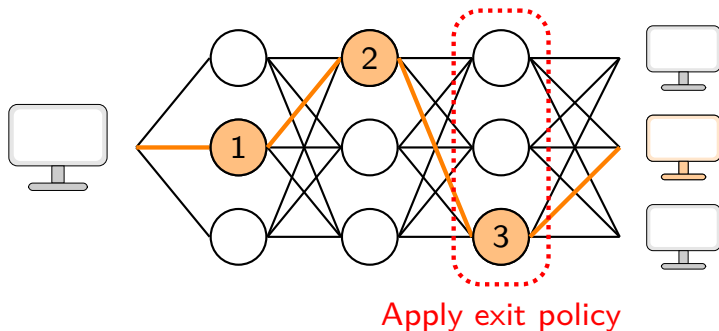
Aurelien Chassagne

April 14, 2026

Table of Content

- Introduction
- General idea
- Simplified version (single authority)
- Multi-authority setup
- Implementation
- Future work

Introduction



Problem: Legitimacy of destination is only verifiable at the **exit node**

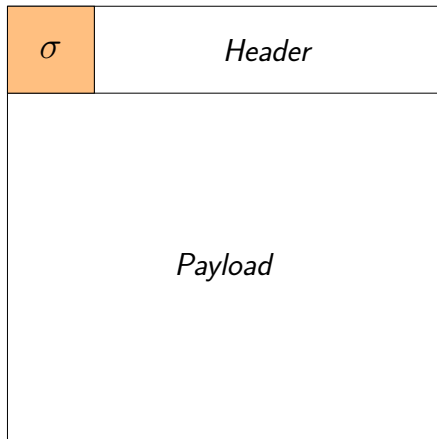
Objective: *Legitimacy of destination verifiable at **any hop** without compromising privacy*

Objective

Idea: Add a **credential** to the packet's header

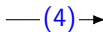
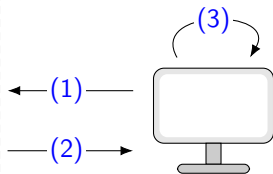
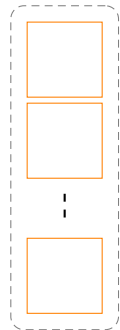
Desired properties:

- Linked to the final destination
- Verifiable at any hop
- Preserve unlinkability
I.e. updated at each hop
- Preserve client anonymity
I.e. cannot be linked to the client
- Preserve client privacy
I.e. do not leak client information

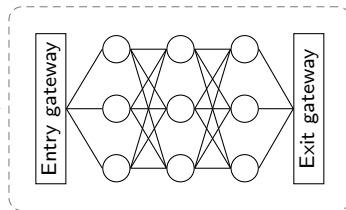


System Architecture

Authorities



Decentralized Network



- (1) Request
- (2) Issue
- (3) Aggregate
- (4) Rerandomize credential

Assumption: Multi-authorities

General Idea (1)

In general, credentials are verified using bilinear pairing such as:

$$e(\sigma, G) == e(M, PK)$$

Credential ← σ M → Message (EC point) PK → Authority's Public Key

General Idea (1)

In general, credentials are verified using bilinear pairing such as:

$$e(\sigma, G) == e(H, PK)$$

Credential $\leftarrow$ σ

H $\rightarrow$ Header chunks containing destination Δ

PK $\rightarrow$ Authority's Public Key

Header:



General Idea (2)

Problem: When packet is processed, the final destination chunk Δ is shifted.

Header at node 1:



Header at node 2:



Header at node 3:



General Idea (3)

Solution: Using all chunks that could contain final destination Δ

$$e(\sigma, G) == e(N'_2 + N'_3 + \Delta', PK)$$

Header:



General Idea (3)

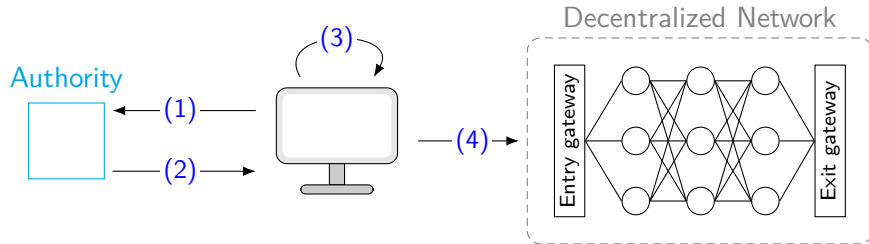
Solution: Using all chunks that could contain final destination Δ

$$e(\sigma, G) == e(N'_2 + N'_3 + \Delta', PK)$$

Header:



Credential - Single Authority (1)



Assumption: Single-authority

Public parameters:

- $PK = yG$
- $\{yN_i\} \forall \text{ mixnode } i$
- $\{yG_{2j-1}\} \forall \text{ hop } j$

where y is the authority secret key

Credential - Single Authority (2)

Credential Construction:

s_i = shared secret *client* - *mixnode*_{*i*}

$$\sigma_1 = y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5)$$

Credential - Single Authority (2)

Credential Construction:

s_i = shared secret *client* - *mixnode* _{i}

$$\begin{aligned}\sigma_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3}_{\text{Public Parameters}} + \underbrace{yN_2}_{\text{Public Parameters}} + (s_1 + s_2 + s_3)\underbrace{yG_1}_{\text{Public Parameters}} + (s_1 + s_2)\underbrace{yG_3}_{\text{Public Parameters}} + (s_1)\underbrace{yG_5}_{\text{Public Parameters}}\end{aligned}$$

Credential - Single Authority (2)

Credential Construction:

s_i = shared secret *client* - *mixnode* _{i}

$$\begin{aligned}\sigma_1 &= y(\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3 + yN_2}_{\text{Public Parameters}} + \underbrace{(s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5}_{\text{Public Parameters}}\end{aligned}$$

Credential Update at each hop:

$$\sigma_{i+1} = \sigma_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1}$$

Credential - Single Authority (2)

Credential Construction:

s_i = shared secret *client* - *mixnode_i*

$$\begin{aligned}\sigma_1 &= y(\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3 + yN_2}_{\text{Public Parameters}} + \underbrace{(s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5}_{\text{Public Parameters}}\end{aligned}$$

Credential Update at each hop:

$$\begin{aligned}\sigma_{i+1} &= \sigma_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1} \\ &= \sigma_i - y \underbrace{\left(s_i(G_1 + G_3 + G_5 + G_7) + N_{i+1} \right)}_{\text{In-Out chunk differences}}\end{aligned}$$

Credential - Single Authority (2)

Credential Construction:

s_i = shared secret *client* - *mixnode_i*

$$\begin{aligned}\sigma_1 &= y(\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3 + yN_2}_{\text{Public Parameters}} + \underbrace{(s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5}_{\text{Public Parameters}}\end{aligned}$$

Credential Update at each hop:

$$\begin{aligned}\sigma_{i+1} &= \sigma_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1} \\ &= \sigma_i - y \underbrace{(s_i(G_1 + G_3 + G_5 + G_7) + N_{i+1})}_{\text{In-Out chunk differences}}\end{aligned}$$

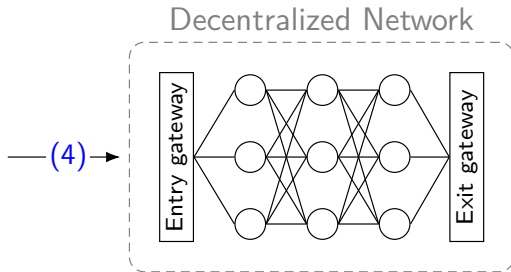
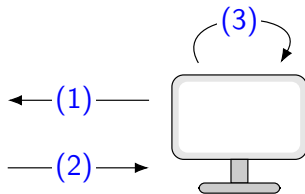
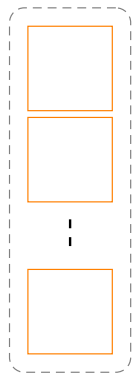
Problem: In a **single-authority** architecture, the authority **breaks** the **unlinkability** property

Multi-Authorities

Problem: In a **single-authority** architecture, the authority **breaks** the **unlinkability** property

Solution: Use a **multi-authorities** architecture (i.e. nobody knows the secret key y)

Authorities



Multi-Authority Setup

Let's explain the setup for n -out-of- K authorities:

Each authority k :

- has a public identifier x_k
- generates a secret polynomial f_k of degree $n - 1$
- sends $f_k(x_{k'})$ to each authority $k' \in K$
- aggregates received values $y_k = \sum_{i \in K} f_i(x_k)$

Credential request:

- Client requires at least n authorities to sign his destination Δ (i.e. $y_k \Delta$)
- Client aggregates into $y \Delta$ using Lagrange interpolation

Implementation

- **Version:** Single-authority (functional prototype)
- **Language:** Python (jupyter notebook)
- **Librairy:** mclbn256 (python wrapper for 'mcl', a fast C++/Assembly pairing-based lib)

↓ library	language	Full pairing (ms)	Miller loop (ms)	Final exponentiation (ms)
mcl (<i>BN_SNARK1</i>)	C++	0.4765	0.2137	0.2628
gnark-crypto(<i>bn254</i>)	Go	0.4890	0.2321	0.2569
constantine (<i>BN254_Snarks</i>)	Nim	0.6471	0.2934	0.3424
kilic/bn254	Go	0.8454	0.3990	0.4464
arkworks(+asm) (<i>bn254</i>)	Rust	1.0655	0.4597	0.6058
arkworks(<i>bn254</i>)	Rust	1.1915	0.5265	0.6649
geth/cloudflare(<i>bn256</i>)	Go	1.3013	0.4874	0.8139
libff(<i>alt_bn128</i>)	C++	2.1000	0.6500	1.4500
geth/google(<i>bn256</i>)	Go	11.4304	4.1681	7.2622

Figure: Pairing benchmark: <https://hackmd.io/@gnark/eccbench>

- Update the code for multi-authorities
- Update overhead costs for multi-authorities
- Draft a secure analysis
- Polishing the multi-authority setup explanation
- Improving credentials features (e.g. blacklist destinations, validity period, ...)

What to do with the previous paper ?

Workload: 1 day / week

- **Benchmark:**

- Drop the NIST benchmark $\Rightarrow$ bit-level randomness not useful anymore
- Add overhead cost/comparison with original Sphinx $\Rightarrow$ Manuel's master thesis

- **Decentralized Path:**

- Security analysis
- Performance overhead
- Smoother integration $\Rightarrow$ difficult

- **Motivation & Application:**

- Why decentralization prevents misbehaving client ? $\Rightarrow$ 'solved' with decentralized path
- Why decentralization helps exit policies ? $\Rightarrow$ difficult... Current work (but without STTPs)

- **Security:**

A reviewer mentioned that our game-based proof was not "formal" but "heuristic" ?!